

Université Versailles Saint-Quentin-en-Yvelines

Master CHPS

# TP PPCS

MPI, Docker Swarm et Benchmarks de Communication

Réalisé par :

**Arab Lina**

Année universitaire : 2025/2026

# Table des matières

|          |                                                                          |          |
|----------|--------------------------------------------------------------------------|----------|
| <b>1</b> | <b>Méthodologie</b>                                                      | <b>3</b> |
| 1.1      | Objectif du TP . . . . .                                                 | 3        |
| 1.2      | Préparation de l'environnement . . . . .                                 | 3        |
| 1.2.1    | Initialisation du Swarm et création du réseau . . . . .                  | 3        |
| 1.2.2    | Déploiement des conteneurs MPI . . . . .                                 | 4        |
| 1.2.3    | Connexion au conteneur head . . . . .                                    | 4        |
| 1.3      | Découverte des adresses IP des conteneurs . . . . .                      | 4        |
| 1.4      | Vérification de MPI via mpi4py . . . . .                                 | 4        |
| 1.4.1    | Commande d'exécution . . . . .                                           | 5        |
| 1.4.2    | Consultation de la configuration de mapping MPI . . . . .                | 5        |
| 1.5      | Compilation et exécution de programmes MPI en C . . . . .                | 5        |
| 1.5.1    | Récupération des sources C . . . . .                                     | 5        |
| 1.5.2    | Compilation des programmes . . . . .                                     | 6        |
| 1.5.3    | Exécution du programme <code>hello_world</code> . . . . .                | 6        |
| 1.5.4    | Exécution du programme <code>any_source</code> . . . . .                 | 6        |
| 1.6      | Compilation et exécution des OSU Micro-Benchmarks . . . . .              | 7        |
| 1.6.1    | Préparation des sources . . . . .                                        | 7        |
| 1.6.2    | Configuration et compilation . . . . .                                   | 7        |
| 1.7      | Surveillance des ressources Docker . . . . .                             | 7        |
| <b>2</b> | <b>Résultats et Analyse</b>                                              | <b>8</b> |
| 2.1      | Validation du fonctionnement MPI . . . . .                               | 8        |
| 2.1.1    | Programme <code>hello_world</code> . . . . .                             | 8        |
| 2.1.2    | Programme <code>any_source</code> . . . . .                              | 9        |
| 2.2      | Benchmarks OSU : latence . . . . .                                       | 9        |
| 2.2.1    | Commande exécutée . . . . .                                              | 9        |
| 2.2.2    | Résultats obtenus . . . . .                                              | 9        |
| 2.2.3    | Analyse des résultats . . . . .                                          | 10       |
| 2.3      | Benchmarks OSU : bande passante bidirectionnelle . . . . .               | 10       |
| 2.3.1    | Commande exécutée . . . . .                                              | 10       |
| 2.3.2    | Résultats obtenus . . . . .                                              | 11       |
| 2.3.3    | Tableau récapitulatif du benchmark C ( <code>osu_bibw</code> ) . . . . . | 11       |
| 2.3.4    | Analyse des résultats . . . . .                                          | 12       |
| 2.3.5    | Comparaison C vs Python . . . . .                                        | 12       |
| 2.4      | Surveillance de la mémoire des conteneurs . . . . .                      | 13       |
| 2.4.1    | Résultat <code>docker stats</code> . . . . .                             | 13       |
| 2.4.2    | Interprétation . . . . .                                                 | 13       |



# Chapitre 1

## Méthodologie

### 1.1 Objectif du TP

L'objectif de ce TP dans le cadre du module *Paradigmes de Programmation pour le Calcul Scientifique (PPCS)* est :

- ▷ de déployer un environnement MPI distribué à l'aide de **Docker Swarm** ;
- ▷ de vérifier le bon fonctionnement de MPI avec des programmes simples en C et en Python ;
- ▷ d'évaluer les performances de communication via les OSU Micro-Benchmarks (latence et bande passante) ;
- ▷ de surveiller les ressources des conteneurs Docker pendant l'exécution.

L'ensemble des commandes, résultats et interprétations sont présentés dans ce rapport.

### 1.2 Préparation de l'environnement

#### 1.2.1 Initialisation du Swarm et création du réseau

Toutes les commandes de cette section sont exécutées sur la machine hôte (Windows, via PowerShell).

##### Initialisation de Docker Swarm

```
docker swarm init
```

Cette commande transforme la machine locale en manager d'un cluster Docker Swarm.

##### Création du réseau overlay

Le réseau utilisé pour MPI est :

```
docker network create --driver=overlay --attachable yml_mpinet
```

Ce réseau `yml_mpinet` est de type *overlay*, permettant aux conteneurs de différents nœuds (logiques) du Swarm de communiquer sur un même sous-réseau virtuel.

### 1.2.2 Déploiement des conteneurs MPI

Le TP fournit un fichier `docker-compose.yml` permettant de lancer :

- ▷ un conteneur `mpihead` (nœud “head”);
- ▷ plusieurs conteneurs `mpinode` (nœuds de calcul).

Le déploiement effectué :

```
docker compose up --scale mpihead=1 --scale mpinode=2 -d
```

Cette commande lance :

- ▷ 1 conteneur `ppcs-cm1-2025-mpihead-1`;
- ▷ 2 conteneurs `ppcs-cm1-2025-mpinode-1` et `ppcs-cm1-2025-mpinode-2`.

### 1.2.3 Connexion au conteneur head

Toutes les commandes MPI suivantes sont exécutées à l’intérieur du conteneur `mpihead` :

```
docker exec -it ppcs-cm1-2025-mpihead-1 bash
```

Le prompt devient alors :

```
mpiuser@48d5830ac016:~$
```

Le conteneur `48d5830ac016` correspond au nœud `mpihead`.

## 1.3 Découverte des adresses IP des conteneurs

Pour exécuter des programmes MPI sur plusieurs conteneurs, il est nécessaire de connaître leurs adresses IP dans le réseau `yaml_mpinet`.

La commande suivante est exécutée **dans le conteneur head** :

```
sudo curl --unix-socket /var/run/docker.sock \
http://localhost/containers/json | \
jq -r 'map(.NetworkSettings[]."yaml_mpinet"."IPAMConfig"."IPv4Address")[]'
```

Les adresses IP obtenues sont :

- ▷ 10.0.1.5
- ▷ 10.0.1.2
- ▷ 10.0.1.4

Dans la suite du TP, deux nœuds de calcul sont utilisés, avec par exemple :

- ▷ 10.0.1.5
- ▷ 10.0.1.4

Le sous-réseau utilisé est donc `10.0.1.0/24`.

## 1.4 Vérification de MPI via `mpi4py`

Une première vérification est effectuée à l’aide du jeu de tests `mpi4py_benchmarks`.

### 1.4.1 Commande d'exécution

Depuis le conteneur head :

```
mpirun --mca orte_base_help_aggregate 0 \  
  --mca btl_tcp_if_include 10.0.1.0/24 \  
  -n 2 \  
  -host 10.0.1.5,10.0.1.4 \  
  python3 ./mpi4py_benchmarks/all_tests.py
```

- ▷ `-n 2` : on lance 2 processus MPI ;
- ▷ `-host 10.0.1.5,10.0.1.4` : les processus sont répartis sur les deux conteneurs de calcul ;
- ▷ `btl_tcp_if_include 10.0.1.0/24` : on force MPI à utiliser l'interface réseau du Swarm.

La commande se termine sans erreur, ce qui valide le bon fonctionnement de MPI sur deux conteneurs via `mpi4py`.

### 1.4.2 Consultation de la configuration de mapping MPI

Pour mieux comprendre la politique de placement des processus MPI, la commande suivante a été exécutée :

```
ompi_info --level 9 --param rmaps base
```

Cette commande affiche de nombreux paramètres du composant `rmaps` (mapping et ranking des processus). On y voit notamment :

- ▷ `rmaps_base_mapping_policy` : politique de mapping (par défaut, par core ou par socket selon `np`) ;
- ▷ `rmaps_base_ranking_policy` : politique de ranking (ordre de numérotation des rangs MPI) ;
- ▷ `rmaps_base_oversubscribe` : permet ou non la surallocation des nœuds.

Cela confirme que les deux processus MPI sont correctement répartis sur les nœuds disponibles.

## 1.5 Compilation et exécution de programmes MPI en C

### 1.5.1 Récupération des sources C

Les sources C fournies dans le cadre du cours sont récupérées depuis le dépôt GitHub `master-mpi`.

```
cd /usr/local/var/mpishare  
git clone https://github.com/jmbatto/master-mpi  
cd master-mpi
```

## 1.5.2 Compilation des programmes

Deux programmes sont utilisés :

- ▷ `hello_world_mpi.cc` : programme classique de test MPI;
- ▷ `mpi_any_source.cc` : test de la réception via `MPI_ANY_SOURCE`.

Compilation :

```
mpicc hello_world_mpi.cc -o hello_world
mpicc mpi_any_source.cc -o any_source
```

Vérification du contenu du répertoire :

```
ls .
LICENSE hello_world mpi41-debian-bullseye
README.md hello_world_mpi.cc mpi_any_source.cc
any_source mpi31-debian-stretch
```

## 1.5.3 Exécution du programme `hello_world`

```
mpirun --mca orte_base_help_aggregate 0 \
  --mca btl_tcp_if_include 10.0.1.0/24 \
  -n 2 \
  -host 10.0.1.5,10.0.1.4 \
  hello_world
```

Résultat obtenu :

```
Hello world from processor 48d5830ac016, rank 0 out of 2 processors
Hello world from processor bf709b430fdf, rank 1 out of 2 processors
```

On observe :

- ▷ deux rangs MPI (0 et 1);
- ▷ exécutés sur deux hôtes différents (deux conteneurs Docker);
- ▷ le message affiche le nom du processeur (ici l'ID du conteneur).

## 1.5.4 Exécution du programme `any_source`

```
mpirun --mca orte_base_help_aggregate 0 \
  --mca btl_tcp_if_include 10.0.1.0/24 \
  -n 2 \
  -host 10.0.1.5,10.0.1.4 \
  any_source
```

Résultat obtenu :

```
[MPI process 0] I send value 12345.
[MPI process 1] I received value 12345, from rank 0.
```

Ce programme illustre l'utilisation de `MPI_ANY_SOURCE` pour recevoir un message sans spécifier explicitement l'émetteur. Dans ce cas, le processus de rang 1 reçoit bien la valeur envoyée par le rang 0.

## 1.6 Compilation et exécution des OSU Micro-Benchmarks

### 1.6.1 Préparation des sources

Dans le répertoire partagé :

```
mpiuser@48d5830ac016:/usr/local/var/mpishare$ ls .
master-mpi osu-micro-benchmarks-7.3.tar.gz
osu-micro-benchmarks-7.3
```

Les sources des OSU Micro-Benchmarks sont déjà extraites dans `osu-micro-benchmarks-7.3`.

### 1.6.2 Configuration et compilation

```
cd /usr/local/var/mpishare/osu-micro-benchmarks-7.3
./configure CC=mpicc CXX=mpicxx
make -j4
```

Les binaires sont générés dans les sous-répertoires, notamment :

- ▷ `c/mpi/pt2pt/standard/osu_latency`
- ▷ `c/mpi/pt2pt/standard/osu_bibw`

## 1.7 Surveillance des ressources Docker

Cette partie est effectuée sur l'hôte (PowerShell). Le but est d'observer l'utilisation mémoire des conteneurs pendant les tests.

```
docker stats --no-stream --format "table {{.Name}}\t{{.Container}}\t{{.MemUsage}}"
```

Résultat obtenu :

```
NAME CONTAINER MEM USAGE / LIMIT
ppcs-cm1-2025-mpihead-1 48d5830ac016 71.11MiB / 3.783GiB
ppcs-cm1-2025-mpinode-1 8d7cac477de5 7.82MiB / 3.783GiB
ppcs-cm1-2025-mpinode-2 bf709b430fdf 4.406MiB / 3.783GiB
```

On remarque que le conteneur `mpihead` consomme plus de mémoire (exécution de `mpirun`, des benchmarks et outils), tandis que les nœuds de calcul restent relativement légers.



# Chapitre 2

## Résultats et Analyse

Dans ce chapitre, nous présentons les résultats obtenus :

- ▷ avec les programmes MPI en C (`hello_world`, `any_source`);
- ▷ avec les benchmarks OSU (`osu_latency`, `osu_bibw`);
- ▷ avec la version Python utilisant `mpi4py` afin de comparer les performances avec l'implémentation C;
- ▷ ainsi que l'observation de la consommation mémoire des conteneurs.

### 2.1 Validation du fonctionnement MPI

#### 2.1.1 Programme `hello_world`

Rappel du résultat

```
Hello world from processor 48d5830ac016, rank 0 out of 2 processors
Hello world from processor bf709b430fdf, rank 1 out of 2 processors
```

Interprétation

- ▷ Deux processus MPI ont été lancés sur deux conteneurs distincts;
- ▷ Chaque processus connaît :
  - ▷ son rang (**rank**) dans le communicateur;
  - ▷ le nombre total de processus (**out of 2 processors**);
  - ▷ le nom de l'hôte (ici, l'ID du conteneur Docker).
- ▷ Ce test confirme que :
  - ▷ l'environnement MPI est correctement configuré;
  - ▷ la communication entre les conteneurs via le réseau overlay `yml_mpinet` fonctionne;
  - ▷ MPI est capable de lancer des processus sur plusieurs hôtes.

## 2.1.2 Programme any\_source

### Rappel du résultat

```
[MPI process 0] I send value 12345.  
[MPI process 1] I received value 12345, from rank 0.
```

### Interprétation

- ▷ Le processus 0 envoie une valeur entière (12345) ;
- ▷ Le processus 1 reçoit cette valeur, et récupère également le rang de l'émetteur (0) ;
- ▷ Le code utilise le mécanisme `MPI_ANY_SOURCE`, ce qui permet de recevoir un message en provenance de n'importe quel rang ;
- ▷ Le bon couplage *send/recv* confirme :
  - ▷ la fiabilité de la communication point-à-point ;
  - ▷ l'absence d'erreur de configuration sur le réseau MPI ;
  - ▷ la compatibilité entre le runtime Open MPI installé dans les conteneurs.

## 2.2 Benchmarks OSU : latence

### 2.2.1 Commande exécutée

```
mpirun --mca orte_base_help_aggregate 0 \  
--mca btl_tcp_if_include 10.0.1.0/24 \  
-n 2 \  
-host 10.0.1.5,10.0.1.4 \  
c/mpi/pt2pt/standard/osu_latency
```

### 2.2.2 Résultats obtenus

```
# OSU MPI Latency Test v7.3  
# Size Latency (us)  
# Datatype: MPI_CHAR.  
1 16.10  
2 13.67  
4 14.65  
8 13.45  
16 15.39  
32 13.52  
64 15.80  
128 17.12  
256 17.30  
512 16.57  
1024 17.15  
2048 19.40  
4096 27.94  
8192 27.82
```

```
16384 36.84
32768 58.22
65536 107.20
131072 95.69
262144 129.92
524288 268.83
1048576 620.90
2097152 989.39
4194304 1697.69
```

### 2.2.3 Analyse des résultats

- ▷ La latence est mesurée en microsecondes ( $\mu s$ ) pour un aller-retour entre deux processus MPI;
- ▷ Pour les petits messages (1 à 1024 octets), la latence reste autour de 13–20  $\mu s$ ;
- ▷ À partir de 4–8 Kio, la latence commence à augmenter plus sensiblement;
- ▷ Pour les très gros messages :
  - ▷ 1 MiB :  $\approx 621 \mu s$ ;
  - ▷ 4 MiB :  $\approx 1698 \mu s$ .

Ce comportement est cohérent avec le modèle de communication vu en cours :

- ▷ la latence totale peut s'écrire approximativement comme :

$$T_{\text{comm}} = \alpha + \beta \times \text{taille du message}$$

où  $\alpha$  est le coût fixe (latence) et  $\beta$  le temps par octet (inversément proportionnel à la bande passante) ;

- ▷ pour les petits messages, le terme  $\alpha$  domine (latence quasi constante) ;
- ▷ pour les grands messages, le terme  $\beta n$  devient prédominant, d'où la croissance quasi-linéaire de la latence.

Le fait d'exécuter ces tests dans des conteneurs Docker, sur un réseau overlay TCP, ajoute une surcharge par rapport à un cluster HPC natif, ce qui explique des latences de l'ordre de dizaines de microsecondes.

## 2.3 Benchmarks OSU : bande passante bidirectionnelle

### 2.3.1 Commande exécutée

```
mpirun --mca orte_base_help_aggregate 0 \
--mca btl_tcp_if_include 10.0.1.0/24 \
-n 2 \
-host 10.0.1.5,10.0.1.4 \
c/mpi/pt2pt/standard/osu_bibw
```

### 2.3.2 Résultats obtenus

```
# OSU MPI Bi-Directional Bandwidth Test v7.3
# Size Bandwidth (MB/s)
# Datatype: MPI_CHAR.
1 0.12
2 0.20
4 0.79
8 1.54
16 2.56
32 6.28
64 9.07
128 24.96
256 48.15
512 88.19
1024 189.81
2048 338.30
4096 516.13
8192 1112.16
16384 1753.76
32768 2082.42
65536 1573.01
131072 2206.16
262144 2206.54
524288 2647.90
1048576 2412.22
2097152 2774.90
4194304 2756.11
```

### 2.3.3 Tableau récapitulatif du benchmark C (osu\_bibw)

Le tableau 2.1 présente un extrait des mesures du benchmark MPI Bi-Directional Bandwidth Test `osu_bibw` exécuté par Arab Lina le 7 décembre 2025 sur l'environnement Docker décrit au chapitre 1 (2 processus MPI répartis sur les conteneurs 10.0.1.5 et 10.0.1.4).

| Taille (bytes) | Bande passante C (MB/s) |
|----------------|-------------------------|
| 8192           | 1112.16                 |
| 65536          | 1573.01                 |
| 131072         | 2206.16                 |
| 262144         | 2206.54                 |
| 524288         | 2647.90                 |
| 1048576        | 2412.22                 |
| 2097152        | 2774.90                 |
| 4194304        | 2756.11                 |

TABLE 2.1 – Extrait des résultats du benchmark `osu_bibw` en C

### 2.3.4 Analyse des résultats

- ▷ Les très petits messages (1–16 octets) donnent une bande passante très faible (moins de 3 MB/s) ;
- ▷ À partir de 1–2 Kio, la bande passante augmente fortement (plus de 300 MB/s) ;
- ▷ Pour des tailles de message de l'ordre de 8 Kio à quelques centaines de Kio, la bande passante dépasse 1–2.5 GB/s ;
- ▷ On observe une forme de saturation autour de 2.5–2.8 GB/s pour les plus gros messages.

Ce comportement est conforme à ce qui est attendu :

- ▷ pour de petites tailles de messages :
  - ▷ le coût fixe par message est dominant ;
  - ▷ la bande passante effective est donc faible, malgré un temps d'envoi très court ;
- ▷ pour des tailles plus grandes :
  - ▷ chaque message transporte beaucoup de données ;
  - ▷ on exploite mieux la capacité du réseau ;
  - ▷ on se rapproche de la bande passante maximale du lien utilisé.

Il faut aussi garder en tête que :

- ▷ les tests sont réalisés dans un environnement virtualisé (Docker) ;
- ▷ la communication passe par un réseau overlay géré par Docker Swarm sur la même machine physique ;
- ▷ ces chiffres ne reflètent pas un réseau HPC InfiniBand, mais ils montrent néanmoins une évolution cohérente en fonction de la taille des messages.

### 2.3.5 Comparaison C vs Python

Nous avons ensuite exécuté la commande suivante avec mpi4py (Python) :

```
mpirun --mca orte_base_help_aggregate 0 \  
--mca btl_tcp_if_include 10.0.1.0/24 \  
-n 2 \  
-host 10.0.1.5,10.0.1.4 \  
python3 ./mpi4py_benchmarks/all_tests.py
```

Cette commande fournit également un test MPI Bi-Directional Bandwidth Test. Voici un extrait des résultats obtenus (Python) :

```
# MPI Bi-Directional Bandwidth Test  
# Size [B] Bandwidth [MB/s]  
8192 294.32  
65536 479.73  
1048576 1039.55  
4194304 1050.89
```

Nous comparons donc ces valeurs avec la version C :

On observe que la variante C est environ deux fois plus rapide que l'implémentation Python. Cela s'explique par la surcharge de l'interpréteur Python et de la surcouche fournie par mpi4py, bien que les deux tests s'appuient finalement sur la même implémentation MPI sous-jacente.

| Taille (bytes) | C (MB/s) | Python (MB/s) |
|----------------|----------|---------------|
| 8192           | 1112.16  | 294.32        |
| 65536          | 1573.01  | 479.73        |
| 1048576        | 2412.22  | 1039.55       |
| 4194304        | 2756.11  | 1050.89       |

TABLE 2.2 – Comparaison C / Python de la bande passante bidirectionnelle

## 2.4 Surveillance de la mémoire des conteneurs

### 2.4.1 Résultat docker stats

```
NAME CONTAINER MEM USAGE / LIMIT
ppcs-cm1-2025-mpihead-1 48d5830ac016 71.11MiB / 3.783GiB
ppcs-cm1-2025-mpinode-1 8d7cac477de5 7.82MiB / 3.783GiB
ppcs-cm1-2025-mpinode-2 bf709b430fdf 4.406MiB / 3.783GiB
```

### 2.4.2 Interprétation

- ▷ Le conteneur `mpihead` consomme le plus de mémoire ( $\approx 71$  MiB) :
  - ▷ il héberge l'utilisateur `mpiuser`, les outils (git, compilateur, etc.);
  - ▷ il lance les commandes `mpirun` et les benchmarks OSU;
- ▷ Les nœuds `mpinode-1` et `mpinode-2` consomment beaucoup moins :
  - ▷ ils exécutent essentiellement les processus MPI esclaves;
  - ▷ leur surcharge mémoire est limitée.
- ▷ Cela illustre la différence de rôle :
  - ▷ le head-node joue le rôle d'interface utilisateur et de lanceur de jobs;
  - ▷ les nœuds de calcul se contentent d'exécuter les rangs MPI.

# Chapitre 3

## Conclusion

Dans ce TP, nous avons mis en œuvre un environnement MPI distribué au-dessus de Docker Swarm, et validé son fonctionnement à l'aide de plusieurs niveaux de tests :

- ▷ **Déploiement et réseau** : initialisation de `docker swarm`, création d'un réseau overlay (`yml_mpinet`) et lancement de conteneurs `mpihead` et `mpinode`. La récupération des adresses IP (`10.0.1.5`, `10.0.1.4`, ...) a permis de cibler précisément les hôtes dans les commandes `mpirun`.
- ▷ **Configuration du runtime MPI** : la commande `ompi_info -level 9 -param rmaps base` a permis d'inspecter la politique de placement et de numérotation des processus, et de vérifier la répartition correcte des rangs sur les différents conteneurs.
- ▷ **Programmes MPI simples** : les programmes `hello_world` et `any_source` ont confirmé la bonne configuration du runtime MPI, la répartition des rangs sur plusieurs conteneurs et le fonctionnement de la communication point-à-point, y compris avec `MPI_ANY_SOURCE`.
- ▷ **Benchmarks OSU en C** :
  - ▷ `osu_latency` a mis en évidence une latence quasi-constante pour les petits messages, puis croissante avec la taille, conformément au modèle  $\alpha + \beta n$  ;
  - ▷ `osu_bibw` a montré une bande passante croissante avec la taille de message, jusqu'à une saturation aux alentours de 2.5 à 2.8 GB/s, typique de l'atteinte de la capacité maximale du lien dans cet environnement.
- ▷ **Benchmarks Python / mpi4py** : l'exécution de `mpi4py_benchmarks/all_tests.py` a fourni des mesures de latence, de bande passante et de bande passante bidirectionnelle dans la version Python, sur la même configuration réseau.
- ▷ **Comparaison C / Python** : les résultats du test MPI Bi-Directional Bandwidth Test provenant de `osu_bibw` (C) et de `mpi4py_benchmarks` (Python) ont été rassemblés dans un tableau commun. Cette comparaison a montré que la surcouche Python et l'interpréteur introduisent une baisse sensible de la bande passante effective par rapport à l'implémentation directe en C, bien que les deux s'appuient sur la même bibliothèque MPI.
- ▷ **Surveillance des ressources** : la commande `docker stats` a permis de visualiser la consommation mémoire de chaque conteneur, confirmant le rôle plus « lourd » du conteneur `mpihead`.

Ce TP illustre concrètement les notions vues en cours :

- ▷ la séparation entre *head node* et nœuds de calcul ;

- ▷ l'importance de la latence et de la bande passante dans les performances des applications parallèles ;
- ▷ l'impact de la taille des messages sur les mesures ;
- ▷ la différence de coût entre une implémentation native en C et une implémentation basée sur Python et `mpi4py` ;
- ▷ la possibilité d'utiliser des environnements conteneurisés (Docker) pour reproduire une architecture de type cluster MPI sur une seule machine.

Des prolongements possibles seraient :

- ▷ tester davantage de processus MPI (`-n 4`, `-n 8`, ...);
- ▷ étudier le comportement des benchmarks OSU pour des collectives (`osu_allreduce`, ...);
- ▷ comparer les résultats obtenus dans Docker avec ceux d'une exécution native sur un cluster HPC ;
- ▷ approfondir la comparaison de performances entre C et Python pour d'autres types de communications (collectives, non bloquantes, etc.).